

Secure JWT Propagation in Microservices

Executive Summary: Securely forwarding user identity (via JWT) between microservices can be achieved by various methods, each with trade-offs. In an internal, trusted network, **simple token forwarding** (propagating the gateway-issued JWT to downstream services) is low-complexity and performant, but requires strict token validation (signature, `aud`, `exp`, etc.) in each service. In more isolated trust domains, **OAuth2 token exchange (RFC 8693)** or **audience-restricted tokens** are safer: the Customer service swaps the original JWT for a new token intended for the Address service (with appropriate `aud` and scopes) [12†L90-L99] [33†L1-L4]. **Token introspection** (RFC 7662) is another option: Address can query the authorization server to verify the token's validity and claims [16†L1-L4]. Finally, **mutual TLS (mTLS) with short-lived certificates or tokens** offers strong authentication without relying solely on bearer tokens [37†L461-L470], at the cost of complex PKI management. The report below analyzes these approaches, discusses threat models, and provides concrete implementation guidance, code samples (Java Spring Boot, Node/Express), gateway and OAuth2-server configurations, and best practices (logging, token lifetime, rotation). A comparison table and sequence diagrams summarize the options.

Propagation Options Overview

- **Simple Token Forwarding:** The API Gateway authenticates the client and attaches a signed JWT (e.g. in `Authorization: Bearer ...`). The Customer service receives this JWT and forwards it unchanged to the Address service. The Address service validates the JWT's signature (using the identity provider's public keys) and claims (issuer, audience, expiration, scopes) [20†L562-L571] [30†L58-L66]. This is the easiest approach and adds no extra network calls, but trusts that all services correctly enforce token validation. If services share a trust boundary (e.g. same VPC/cluster), this may suffice [37†L473-L481].
- **OAuth2 Token Exchange (RFC 8693):** The Customer service treats the gateway's JWT as a **subject token** and calls the OAuth2 provider's token-exchange endpoint, requesting a new token for the Address service. The request includes `grant_type=urn:ietf:params:oauth:grant-type:token-exchange`, `subject_token=<jwt>`, `subject_token_type=urn:ietf:params:oauth:token-type:access_token`, and optionally `audience=<address-service-client-id>` [12†L120-L128] [33†L1-L4]. The OAuth2 server verifies the original token and returns a new JWT whose `aud` claim is set to the Address service (and possibly with reduced scopes) [12†L202-L211]. Customer then calls Address using this new token. Token Exchange prevents misuse of the original token (scoping it narrowly) and clearly separates trust domains, at the cost of an extra token request.
- **Audience-Restricted Tokens:** Here the API Gateway issues JWTs that include the Address service's audience from the start (either as a separate token or multi-audience JWT). For example, the Gateway could issue a JWT with `"aud":["customer-service","address-service"]`. Customer forwards the same JWT to Address; Address checks that its own identifier is present in `aud` [20†L562-L571]. This avoids a separate exchange call but requires the Gateway/IDP to know downstream audiences in advance. It can also be applied in Token Exchange (the new token's `aud` is set) or via client scopes (Keycloak's audience mapper).
- **Token Introspection (RFC 7662):** Instead of trusting the JWT offline, the Address service can call the authorization server's introspection endpoint with the received JWT. The introspection

request (typically HTTP Basic or bearer auth) includes `token=<jwt>`. The server responds with a JSON indicating `active: true/false` and the token's metadata (claims) if active [16†L1-L4]. This allows Address to verify tokens dynamically (including revocation status), but adds latency and requires the auth server to be reachable. Introspection is mainly used for opaque tokens, but can be used for JWTs if constant revocation checking is needed.

- **mTLS + Short-Lived Tokens:** Instead of (or in addition to) bearer tokens, use mutual TLS between services. Each microservice holds a client certificate and validates peers' certs on each HTTPS call [37†L461-L470]. A short-lived service token can be obtained via mTLS (e.g. using OAuth2 JWT-bearer with TLS client auth) and presented as a Bearer token. mTLS provides strong identity binding and confidentiality, but requires managing a PKI or service mesh certificates, and addressing certificate rotation/revocation [37†L461-L470].

Each option trades complexity vs security vs performance (see comparison table below). In a fully-trusted internal cluster, simple forwarding with strict JWT validation (aud, iss, signature, `exp`, scopes) may be acceptable. When calling services in another domain or needing least-privilege, token exchange or audience-limited tokens are safer (preventing token replay or over-broad use). If tokens must be short-lived or revocable, introspection or mTLS can be used.

Threat Models and Assumptions

Trust Boundaries: We assume microservices run in a controlled network, but do not assume complete trust between them. For instance, "Customer" could be compromised and shouldn't be able to call "Address" without proper authorization. If all services run in the same Kubernetes cluster/VPC, network traffic is isolated; but if services span networks (or if multitenancy is involved), treat each as separate trust domains. The API Gateway is the ultimate entry point (trusts client auth) and issues JWTs (the source of truth for identity).

Threats: An attacker might steal a JWT (e.g. via a compromised service or man-in-the-middle) and replay it to another service. They might use a token issued to Customer service to call Address if audience is not checked. They could also attempt to misuse scopes or impersonate privileged clients. Insider threats (malicious microservice) could forward tokens to unintended services. Our mitigations: always validate the JWT signature, require the correct `aud` claim, use short expiration (`exp`) to limit token reuse, and employ scope checks. Token exchange/limited audience prevents a token meant for Customer being accepted by Address unless specifically exchanged. Introspection detects revoked tokens.

Recommended Defaults: For a typical same-trust environment, the default may be to forward the Gateway's JWT and have Address validate it. Ensure the JWT contains appropriate claims (`iss`, correct `aud`, minimal `scope`). If there is any boundary (e.g. external trust or multi-tenant scenario), use token exchange or mTLS. We recommend always enabling audience restrictions (no wildcard audiences) and short token lifetimes. For example, assume Customer and Address share a trust domain; the default is **simple forwarding** with `aud=address-service` required. If they are in different domains or if Customer should not reuse its JWT, use **token exchange** (Gateway→Customer→AuthZ Server→Customer→Address) [12†L90-L99] [33†L1-L4]. (The API Gateway could also issue separate tokens per service, but token exchange is more flexible with dynamic audiences.) Finally, for the highest security (e.g. microservices in untrusted networks), mutual TLS can enforce service identity [37†L461-L470].

Implementation: Calling Address from Customer

Below we outline steps, HTTP examples, and JWT claims for each main approach. In all cases, Customer sends an HTTP request to Address with the `Authorization: Bearer <token>` header. Address must validate the incoming JWT and handle errors (typically respond 401 Unauthorized or 403 Forbidden on failure).

1. Simple Forwarding

At API Gateway: Issue a JWT for the client upon authentication. Example claims might include:

```
iss: https://auth.example.com
sub: user123
aud: [ "customer-service", "address-service" ]
scope: "address:read address:write"
exp: <soon, e.g. 5min from now>
iat: <now>
jti: <unique-id>
```

(Alternatively, issue separate tokens per service with single-element `aud`.) Attach `Authorization: Bearer <JWT>`.

Customer Service: Receives request with header `Authorization: Bearer <JWT>`. Forwards to Address, preserving the header:

```
GET /address/api/data HTTP/1.1
Authorization: Bearer eyJhbGciOiJSUzI1NiIsIn...
Host: address-service.local
```

(Code snippet below will show this.)

Address Service Validation: On receiving the request, Address extracts the JWT from the `Authorization` header. It performs:

- Verify JWT signature (using public key from issuer's JWKS).
- Check `iss` is trusted, and `aud` contains `"address-service"` [\[20†L562-L571\]](#) . (If missing or different, reject.)
- Check `exp` and `nbf` / `iat` per RFC7519 [\[20†L576-L584\]](#) [\[20†L604-L612\]](#) .
- Check `scope` includes required permission (e.g. `address:read`) or check `azp` (authorized party) if present.
- Optionally check `sub` and custom claims.

If all checks pass, process request; else return `401 Unauthorized` (invalid token) or `403 Forbidden` (valid token but insufficient scope). For example:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json
```


(Include e.g. `address:read` scope claims; `azp` may show the Customer client.)

Address Validation: Same as above (verify signature, check `aud="address-service-client-id"`, `iss`, expiration, and scope). Because token was issued specifically for Address, we know the requester was authorized to get it.

Error Handling: If token exchange fails (400 from auth server), Customer returns an error (e.g. 502 Bad Gateway) to API Gateway. If Address rejects token, return 401 as above.

3. Audience-Restricted Tokens

This can be viewed as a variant of forwarding: the Gateway could issue tokens that already include Address's audience (as in example 1), or Customer could request a new token (similar to exchange) but without explicit grant (simply use identity provider with audience parameter).

Gateway (or AuthZ Server) Config: Configure the access token to include `aud=address-service-client-id`. In Keycloak, for example, one can add the Address client as an "audience" via a client scope mapper.

Customer→Address: Same flow as simple forwarding, since token was pre-crafted. Address's validation (as above) will succeed on audience.

4. Token Introspection

Customer→Address: Forward the received JWT as-is (as in simple forwarding).

Address: Instead of validating locally, call the introspection endpoint:

```
POST /oauth2/introspect
Authorization: Basic {resource_server_credentials}
Content-Type: application/x-www-form-urlencoded

token=<received-JWT>
```

The auth server replies:

```
{
  "active": true,
  "iss": "https://auth.example.com",
  "sub": "user123",
  "aud": ["address-service"],
  "exp": 1625900000,
  "iat": 1625899640,
  "scope": "address:read"
}
```

If `"active": false` or missing, reject the call (401). Otherwise, check the returned claims (especially `aud`, `scope`, `exp`) as above.

Notes: Introspection avoids local signature verification and lets Address see up-to-date token status (e.g. revoked). However, it doubles request cost and requires secure channel to auth server. Use it if token revocation must be honored; otherwise offline JWT validation (above) is faster.

5. Mutual TLS (mTLS) + Short-Lived Token

Setup: Each service has a client certificate (from an internal CA). Configure Address's server to **require** client certificates (e.g. in NGINX or service mesh). Optionally, have Customer obtain a short-lived JWT via client-credentials grant (with TLS client auth) to include as Bearer.

Customer→Address: The HTTP request is done over TLS with client cert. For example, in Java use `HttpClient` with keystore; in Node, use `https.Agent` with `cert` / `key`. The TLS handshake itself authenticates Customer to Address. Optionally include a token:

```
GET /address/api/data HTTP/1.1
Authorization: Bearer eyJhbGciOiJSUzI1NiIsIn...
Host: address-service.local
```

(This token could be blank if cert is sole credential, or a short “one-time” token from an internal STS [37†L461-L470] .)

Address Validation: First, verify the client's TLS certificate is trusted (check chain). From the cert, extract identity (e.g. URI Subject DN or SPIFFE ID) and map to a service identity (e.g. “customer-service”). Then verify the JWT if used: check signature/aud, or if token is one-time, mainly check it's unexpired and matches the client's identity.

Error Handling: If TLS handshake fails or client cert is invalid, the request is terminated. If using token, invalid tokens produce 401 as above. mTLS prevents stolen tokens being reused off the wire, but tokens may not even be needed if mutual auth is enforced.

Code Examples

Below are sample snippets illustrating token propagation and validation in Java (Spring Boot) and Node/Express. These assume standard JWT libraries and HTTP clients.

Java (Spring Boot) – Forwarding and Validation

```
// CustomerService: forwarding JWT to Address using RestTemplate
@RestController
public class CustomerController {

    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/customer/api/data")
    public ResponseEntity<String>
    getDataFromAddress(@RequestHeader("Authorization") String authHeader) {
        // authHeader = "Bearer <token>"
    }
}
```

```

        HttpHeaders headers = new HttpHeaders();
        headers.set("Authorization", authHeader);
        HttpEntity<Void> requestEntity = new HttpEntity<>(headers);
        // Call Address microservice
        ResponseEntity<String> response = restTemplate.exchange(
            "https://address-service.local/address/api/data",
            HttpMethod.GET, requestEntity, String.class);
        return ResponseEntity.ok(response.getBody());
    }
}

// AddressService: validating incoming JWT (offline)
@RestController
public class AddressController {

    @Value("${jwt.jwks-uri}")
    private String jwksUri; // JWKS URL of auth server

    @GetMapping("/address/api/data")
    public ResponseEntity<String> serveData(@RequestHeader("Authorization")
String authHeader) {
        String token = authHeader.substring("Bearer ".length());
        try {
            // Using Nimbus JOSE JWT library as example
            JWKSet jwkSet = JWKSet.load(new URL(jwksUri));
            JWSVerifier verifier = new RSASSAVerifier((RSAKey)
jwkSet.getKeyByKeyId("key1"));
            SignedJWT jwt = SignedJWT.parse(token);
            if (!jwt.verify(verifier)) {
                throw new RuntimeException("Invalid signature");
            }
            JWTClaimsSet claims = jwt.getJWTClaimsSet();
            // Validate issuer, audience, exp, scope
            if (!"https://auth.example.com".equals(claims.getIssuer())) {
                throw new RuntimeException("Invalid issuer");
            }
            if (!claims.getAudience().contains("address-service")) {
                throw new RuntimeException("Invalid audience");
            }
            if (claims.getExpirationTime().before(new Date())) {
                throw new RuntimeException("Token expired");
            }
            // Additional checks (scope, jti, etc.) here...
            // Serve data
            return ResponseEntity.ok("Protected Address data for user " +
claims.getSubject());
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                .body("{\"error\":\"invalid_token\",\"error_description\":\
\"" + e.getMessage() + "\"}");
        }
    }
}

```

```
}  
}
```

Node.js (Express) – Forwarding and Validation

```
// Customer Service (Node.js/Express)  
const express = require('express');  
const axios = require('axios');  
const app = express();  
  
app.get('/customer/api/data', async (req, res) => {  
  const auth = req.header('Authorization');  
  if (!auth) return res.status(401).json({ error: 'no_token' });  
  try {  
    // Forward the JWT to Address service  
    const addressResp = await axios.get('https://address-service.local/  
address/api/data', {  
      headers: { Authorization: auth }  
    });  
    res.json(addressResp.data);  
  } catch (err) {  
    // Propagate error  
    res.status(err.response?.status || 500).send(err.response?.data ||  
err.message);  
  }  
});  
  
// Address Service (Node.js/Express with jsonwebtoken)  
const jwt = require('jsonwebtoken');  
const jwksClient = require('jwks-rsa');  
const client = jwksClient({ jwksUri: 'https://auth.example.com/.well-known/  
jwks.json' });  
  
function getKey(header, callback){  
  client.getSigningKey(header.kid, (err, key) => {  
    if (err) return callback(err);  
    const publicKey = key.getPublicKey();  
    callback(null, publicKey);  
  });  
}  
  
app.get('/address/api/data', (req, res) => {  
  const auth = req.header('Authorization');  
  if (!auth) return res.status(401).json({ error: 'no_token' });  
  const token = auth.replace('Bearer ', '');  
  jwt.verify(token, getKey, {}, (err, payload) => {  
    if (err) {  
      return res.status(401).json({ error: 'invalid_token',  
error_description: err.message });  
    }  
  })  
});
```



```

// Validate claims
if (payload.iss !== 'https://auth.example.com'
    || !payload.aud.includes('address-service')
    || payload.exp < Math.floor(Date.now()/1000)) {
    return res.status(401).json({ error: 'invalid_token',
error_description: 'claims check failed' });
}
// Token is valid
res.json({ data: `Protected address data for user ${payload.sub}` });
});
});

```

The above code illustrates **token forwarding** and local validation. For **token exchange**, the Customer code would first make an HTTP POST to the auth server (similar to Keycloak example 【12†L120-L128】), then use the returned token (instead of the original) when calling Address. For **introspection**, the Address code would perform an HTTP POST to the `/introspect` endpoint (using e.g. `axios.post`) and examine the JSON result instead of decoding the JWT itself. (OIDC/OAuth libraries often have built-in support for these.)

Gateway and OAuth2 Server Configuration Examples

Below are brief examples of how to configure common API gateways and identity providers for JWT validation, audience restrictions, or token exchange.

- **Kong Gateway:** Use the [JWT Authentication plugin](#) to validate incoming JWTs at the gateway. For example, enable it on a service and configure the `issuer` and `jwtks_uri`. Kong will then reject requests without valid JWTs. Kong doesn't natively perform OAuth2 token exchange, but you could implement a custom plugin or use Kong's OAuth2 plugin for client credentials. (Kong does allow passing through the JWT in headers to upstream services.)
- **AWS API Gateway (HTTP API) – JWT Authorizer:** AWS HTTP APIs support JWT authorizers. In the API definition, set up a JWT authorizer with the identity source (e.g. `Authorization` header), issuer (`iss` URL) and audience (the Auth0/Cognito App client ID) 【30†L58-L66】. AWS will then validate the token's signature, `iss`, `aud` / `client_id`, and expiration automatically. For example, using CloudFormation:

```

Resources:
  MyJWTAuthorizer:
    Type: AWS::ApiGatewayV2::Authorizer
    Properties:
      ApiId: !Ref Api
      Name: JwtAuth
      AuthorizerType: JWT
      IdentitySource: ['$request.header.Authorization']
      JwtConfiguration:
        Issuer: https://cognito-idp.${AWS::Region}.amazonaws.com/${UserPoolId}

```

```
Audience:
- !Ref AppClientId
```

When configured, API Gateway will only accept JWTs whose `iss` and `aud` match `["30†L58-L66"]` .

- **NGINX:** Use `ngx_http_auth_jwt_module` (or `auth_jwt` directive) to verify JWTs at the gateway or service proxy. For example:

```
server {
    listen 443 ssl;
    location / {
        auth_jwt "closed site";
        auth_jwt_key_file /etc/nginx/public.key;
        auth_jwt_validate_issuer on;
        auth_jwt_issuer "https://auth.example.com";
        auth_jwt_audience "address-service";
        proxy_pass http://address-service;
    }
}
```

This ensures NGINX rejects requests with invalid or wrong-audience tokens before they reach the service.

- **Istio:** Define a `RequestAuthentication` policy for the Address workload. For example:

```
apiVersion: security.istio.io/v1beta1
kind: RequestAuthentication
metadata:
  name: address-jwt
spec:
  selector:
    matchLabels:
      app: address-service
  jwtRules:
    - issuer: "https://auth.example.com"
      jwksUri: "https://auth.example.com/.well-known/jwks.json"
      audiences: ["address-service"]
```

Istio will then validate incoming JWTs' `iss` and ensure the `aud` claim includes `address-service` `["28†L727-L734"]` . An `AuthorizationPolicy` can additionally enforce scopes or roles.

- **Keycloak (OAuth2 Server):** To use token exchange, enable the `token-exchange-standard` feature and give the Customer client (the one doing exchange) the "exchange token" scope. In Keycloak Admin Console: under Clients > Customer > Client Scopes, enable "Token Exchange". Also enable "Allow Token Exchange" under the Customer client's advanced settings. Then Customer can call:

```
POST /realms/myrealm/protocol/openid-connect/token
grant_type=urn:ietf:params:oauth:grant-type:token-exchange
&subject_token=<origJWT>
&requested_token_type=urn:ietf:params:oauth:token-type:access_token
&audience=address-service-client-id
```

(As shown in Keycloak docs [12†L120-L129] [12†L202-L211] .)

- **Auth0:** Configure a [Custom Token Exchange](#) action for the Application. Auth0 requires that the `subject_token` be validated in custom code. The Gateway or Customer can call Auth0's `/oauth/token` with `grant_type=urn:ietf:params:oauth:grant-type:token-exchange` (similar to RFC8693), if the Tenant has that feature enabled [14†L79-L87] . For audience restrictions, Auth0 issues tokens for a single audience (API identifier) per access token; make sure the Address service's API identifier is set as the `audience` parameter when initially requesting the token from Auth0 (or do exchange to target that audience).

These examples show how to enforce the right token at the gateway or issue the right token for downstream calls. In all cases, the core idea is to ensure the Address service only accepts JWTs explicitly meant for it, and to propagate identity securely through headers.

Logging, Monitoring, and Token Policies

- **Logging:** Log all authentication events and failures. Each microservice should log when it receives a request with a JWT, and whether the token validation succeeded or failed (with reason). Include the token's `jti` and `sub` in audit logs. Generate a **correlation ID** for each request (see OWASP: "correlation ID... helps group log messages to investigate them" [37†L557-L561]). Propagate this ID in headers (e.g. `X-Request-ID`) so logs across services can be correlated. Do not log sensitive token contents (PII), only IDs.
- **Monitoring:** Track metrics like number of rejected tokens, validation errors per endpoint, and token exchange failures. Alerts on spikes of invalid tokens (might indicate attack). Monitor auth server health.
- **Key and Token Rotation:** Rotate signing keys periodically and use the `kid` header. Auth servers (Keycloak, Auth0) support JWKS key rotation. Cache public keys (e.g. AWS Gateway caches 2h [30†L47-L50]) and refresh.
- **Token Lifetime & Scope Policies:** Use short JWT lifetimes (e.g. 5–15 minutes) to limit replay. For highly sensitive calls, even shorter. Limit scopes to minimum needed ("least privilege"). For example, the Gateway could issue the Customer service's token with only `scope: address:read` if only reading is needed, reducing risk if the token leaks. Consider separate scopes for internal (service-to-service) vs user-level calls. Enforce `exp` strictly (allow 1–2 minute clock skew). Optionally use `nbf` (not before) to delay token use if needed. Use `jti` for each token so you could blacklist compromised tokens (in introspection list) if needed.
- **Sample Policy:** *"Access tokens are valid for 5 minutes, with refresh via Client Credentials or token exchange. Each token's `aud` must match the intended service. Scopes are service-specific (e.g. `address:read` , `address:write`). No single token may be used for multiple services unless explicitly multi-audience. Rotation of signing keys occurs weekly, with overlap to allow validation."*

Comparison of Approaches

Approach	Complexity	Security Level	Performance Impact	Infra Required	Use Cases
Token Forwarding	Low – just pass headers	Moderate – relies on strict JWT checks; risks if trust domain is broken	Low – no extra calls	Standard HTTP/TLS only	Trusted internal calls; high performance needed
Audience-Restricted Token	Low – configure IDP or gateway for audience	Moderate-High – ensures token intended for receiver 【20†L562-L571】	Low – similar to forwarding	IDP support for multi-audience	Internal calls with known audiences
Token Exchange (RFC 8693)	Medium – requires STS endpoint call	High – issues service-scoped token, reduces misuse 【12†L90-L99】	Medium – one extra network round-trip	OAuth2/SToken Service (Keycloak/Auth0)	Cross-domain or multi-tenant, least-privilege needed
Introspection (RFC 7662)	High – calls auth server per request	High – checks revocation/active status 【16†L1-L4】	High – network call on each validation	Auth server with introspect API	Opaque tokens, dynamic revocation scenarios
mTLS + Short-Lived Token	High – PKI/mTLS setup	Very High – strong crypto auth 【37†L461-L470】	Low – TLS handshake overhead	PKI (certs), service mesh or TLS setup	Mission-critical services, zero-trust segments

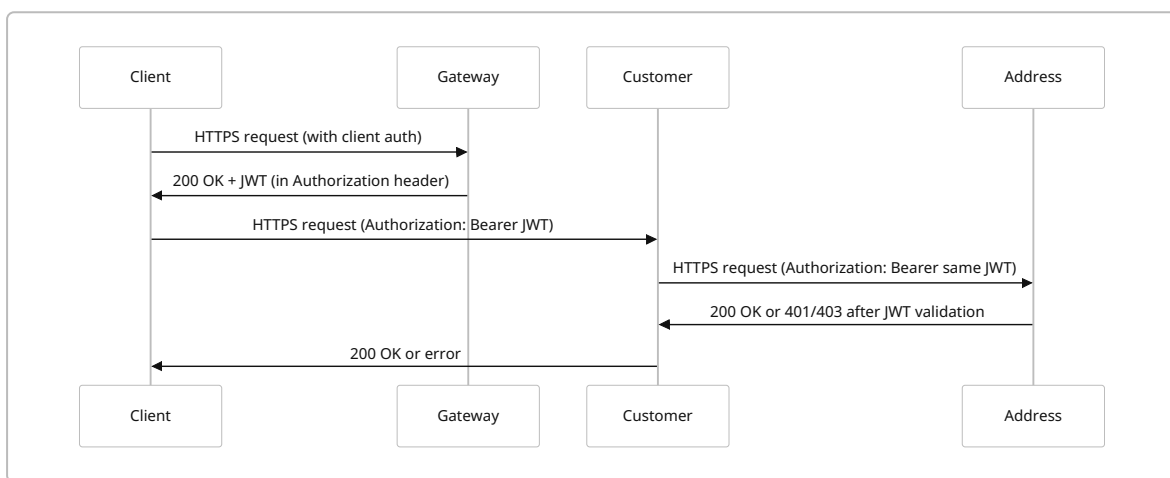
- **Complexity:** Forwarding is simplest (no extra components). Token exchange and introspection need an STS (OAuth2 server) interaction. mTLS requires PKI setup.
- **Security:** mTLS and exchange give the strongest assurance (narrow audience, short tokens, identity proof). Forwarding is weaker if tokens are long-lived or broadly scoped. Introspection ensures up-to-date status but depends on the introspection endpoint's security.
- **Performance:** Forwarding is fastest. Token exchange/introspection incur extra hops. mTLS adds TLS handshake costs (but often caches sessions).

- **Infrastructure:** Forwarding only needs existing identity server for JWT signing. Token exchange/introspection need an OAuth2 token endpoint and introspection endpoint. mTLS needs certificate management.
- **Use Cases:** Use forwarding for same-trust environment and simple setups; audience-restricted tokens as a lightweight improvement. Use token exchange for calls crossing trust boundaries or requiring delegation semantics [12†L90-L99] . Introspection is fit for strict revocation needs (e.g. logout). mTLS suits highly secure or regulated environments where mutual authentication is required [37†L461-L470] .

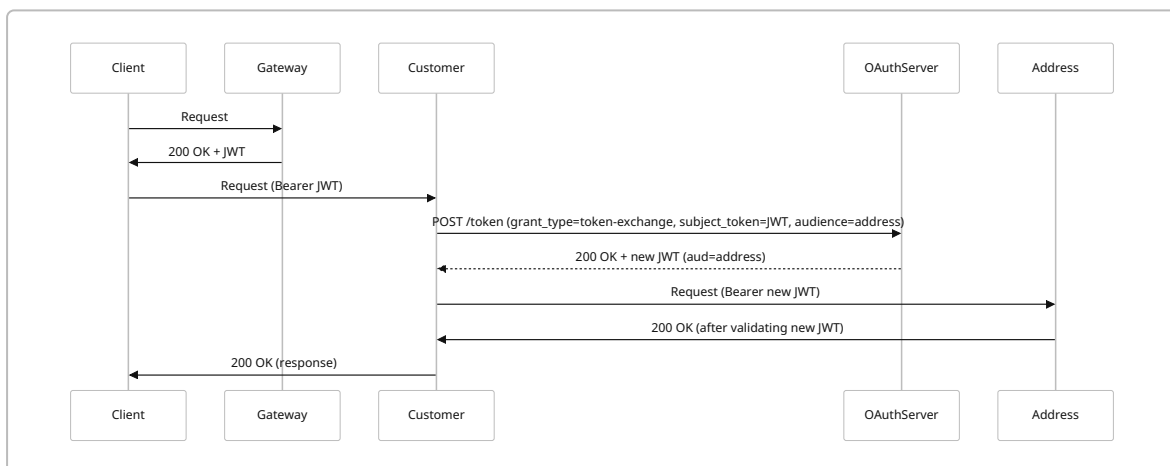
Request Flow Diagrams

Below are mermaid sequence diagrams illustrating each pattern:

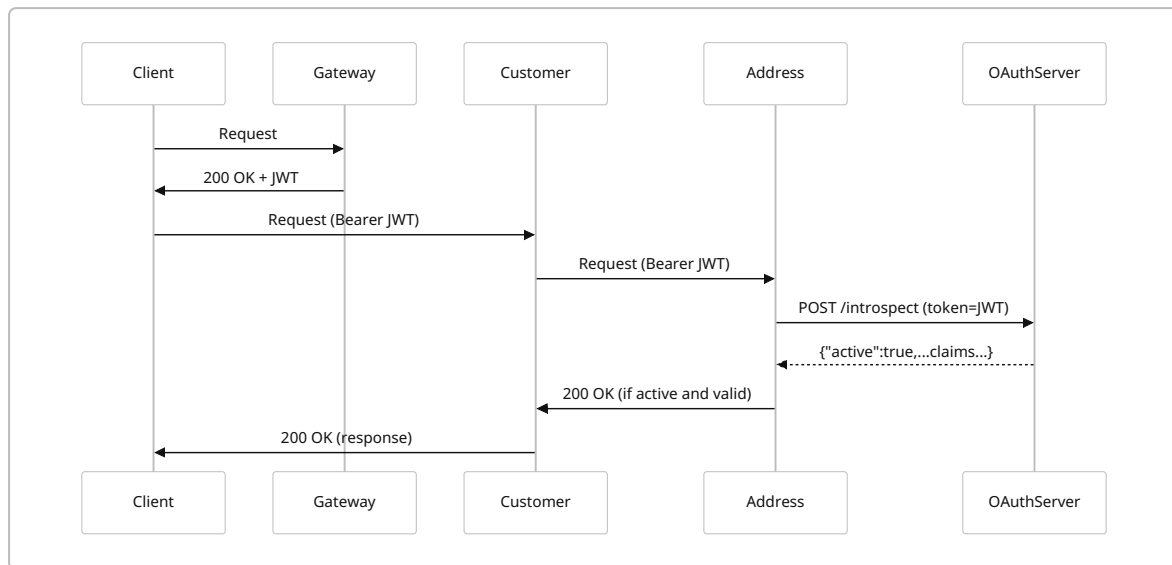
Simple Forwarding (JWT Relay)



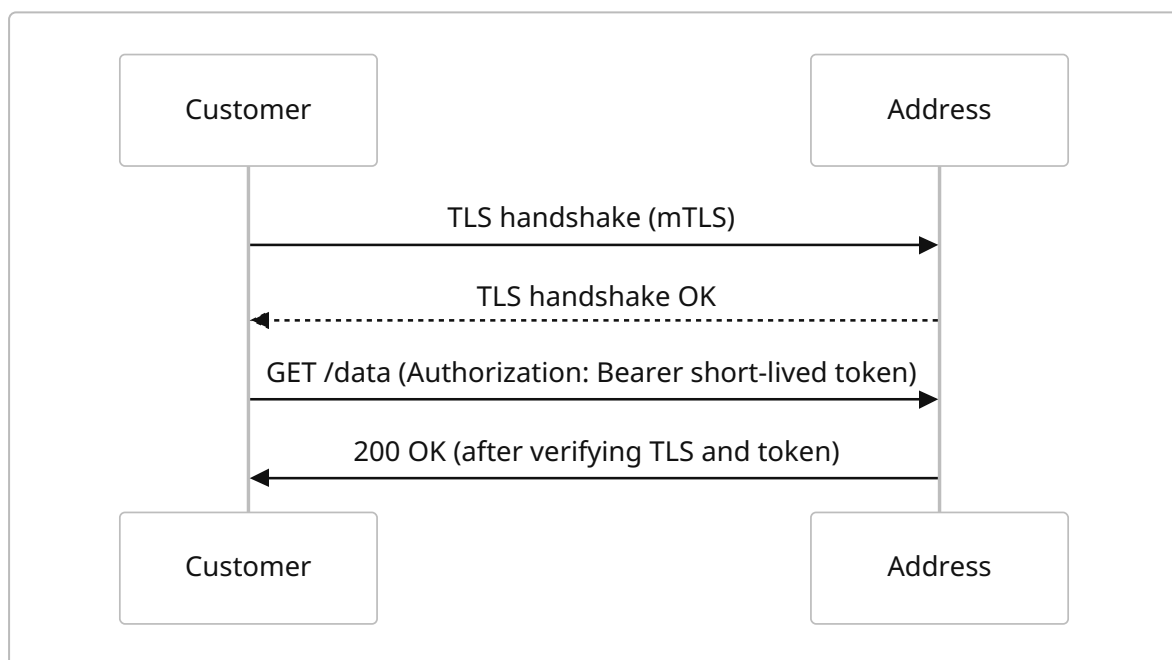
Token Exchange (RFC8693)



Introspection (RFC7662)



mTLS + Short Token



References

- JSON Web Token (JWT) specification [20†L562-L571] [20†L611-L620] . See RFC 7519 for standard JWT claims (issuer `iss`, subject `sub`, audience `aud`, expiration `exp`, issued-at `iat`, JWT ID `jti`) and validation rules. Each service must verify the JWT signature and that its own service name appears in the `aud` claim, else reject [20†L562-L571] .
- OAuth 2.0 Token Exchange (RFC 8693) [12†L90-L99] [33†L1-L4] . Defines the `urn:iETF:params:oauth:grant-type:token-exchange` flow for exchanging one token for another with a different audience or scope. Keycloak and Auth0 have implementations of this exchange [12†L120-L128] [14†L79-L87] .

- OAuth 2.0 Token Introspection (RFC 7662) 【16†L1-L4】 . Allows a service to check with the auth server whether a token is active and retrieve its meta-data.
 - AWS API Gateway JWT Authorizer 【30†L58-L66】 . Validates JWT `iss`, `aud` (or `client_id`), `exp`, etc.
 - Istio RequestAuthentication 【28†L675-L684】 【28†L727-L734】 . Specifies acceptable JWT issuers and audiences for service proxies.
 - OWASP Microservices Security Cheat Sheet 【37†L461-L470】 【37†L473-L481】 . Discusses mTLS (each service has key pair; strong identity, but PKI overhead) and token-based auth (offline vs online validation). Also recommends logging practices like correlation IDs 【37†L557-L561】 .
-